

PROYECTO FINAL ANALISIS Y DISEÑOS DE ALGORITMOS

Katherine Arboleda Ocoro - 202478504
Cristian Camilo Cortes Ortiz - 202478542

Docente: Daniel Quintero Capera
Asignatura: Análisis y Diseños de Algoritmos

Universidad del Valle
Facultad de ingeniería
Escuela de ingeniería de sistemas y computación
Tecnología en desarrollo de software

Univalle, Zarzal

2026

Cada componente tiene una responsabilidad bien definida: el extractor de features no depende del MLP, el MLP no depende de FastAPI, y así sucesivamente. Esto nos permitió desarrollar y probar cada módulo de forma independiente.

RA1 — Red Neuronal Multicapa (MLP) desde cero

¿Por qué implementar el MLP desde cero?

Una de las decisiones más importantes del proyecto fue no usar scikit-learn ni ninguna librería de machine learning para el modelo principal. Queríamos entender de verdad cómo funciona una red neuronal: cómo fluye la información hacia adelante, cómo se calcula el error, cómo se propaga hacia atrás y cómo se actualizan los pesos. Hacer todo eso con NumPy nos obligó a razonar matemáticamente cada paso, lo cual fue exigente pero muy valioso.

Arquitectura de la red

La red tiene la siguiente estructura de capas:

Capa	Tipo	Neuronas / Tamaño	Función de activación
Entrada	Input	20 (features del código)	—
Capa Oculta 1	Dense	128 neuronas	ReLU
Capa Oculta 2	Dense	64 neuronas	ReLU
Salida	Dense	7 neuronas (una por clase)	Softmax

Los 20 parámetros de entrada corresponden a características estructurales extraídas del código fuente mediante el módulo `feature_extractor.py`. La salida de 7 neuronas corresponde a las 7 clases de complejidad, y la neurona con mayor valor es la predicción del modelo.

Hiperparámetros del modelo

Los hiperparámetros los definimos después de varias iteraciones de prueba y error. Los valores finales fueron los siguientes:

Hiperparámetro	Valor	Justificación
Learning Rate inicial	0.005	Más bajo que el default para evitar oscilaciones
LR Decay Factor	0.5 cada 100 épocas	Reduce la tasa para afinar la convergencia
Momentum (SGD)	0.9	Acelera el entrenamiento y reduce oscilaciones
Épocas máximas	600	Se usa Early Stopping para parar antes si es posible
Early Stopping (patience)	100 épocas	Si no mejora en 100 épocas seguidas, detiene
Batch Size	8	Mini-batch para mayor estabilidad que SGD puro
Gradient Clipping	1.0	Evita gradientes explosivos
Ruido de augmentación	$\sigma = 0.015$	Gaussiano suave para balancear clases

Funciones matemáticas implementadas

A continuación, describimos las tres funciones matemáticas principales que implementamos manualmente.

ReLU (Rectified Linear Unit) es la función de activación de las capas ocultas. Su fórmula es sencilla: deja pasar los valores positivos tal cual y convierte en cero los negativos. Esto introduce no-linealidad en la red sin ser costosa computacionalmente:

$$\text{ReLU}(x) = \max(0, x)$$

```
def relu(x):
    return np.maximum(0, x)
```

Softmax se usa en la capa de salida para convertir los valores brutos (logits) en probabilidades que sumen 1. Se le resta el máximo antes de aplicar exp para evitar overflow numérico, que es un truco estándar de estabilidad numérica:

$$\text{Softmax}(x_i) = \exp(x_i) / \sum \exp(x_j)$$

```
def softmax(x):
    e = np.exp(x - np.max(x))
    return e / e.sum()
```

El Gradient Clipping evita que los gradientes crezcan demasiado durante el backpropagation. Si la norma supera un umbral (clip = 1.0), se escala proporcionalmente:

```
def clip_grad(g, clip=1.0):
    norm = np.linalg.norm(g)
```

```

    if norm > clip:
        return g * clip / norm
    return g

```

Forward Propagation

El forward pass calcula la salida de la red dado un vector de entrada. Para cada capa se aplica una transformación lineal seguida de su función de activación:

```

h1_in  = X @ W1 + b1          # Capa oculta 1 (antes de activación)
h1_out = ReLU(h1_in)          # Aplicar ReLU
h2_in  = h1_out @ W2 + b2     # Capa oculta 2 (antes de activación)
h2_out = ReLU(h2_in)          # Aplicar ReLU
o_in   = h2_out @ W3 + b3     # Capa de salida (logits)
o_out  = Softmax(o_in)         # Probabilidades finales

```

Los pesos W_1 , W_2 , W_3 tienen dimensiones (20×128) , (128×64) y (64×7) respectivamente. Los sesgos b_1 , b_2 , b_3 tienen dimensiones $(128,)$, $(64,)$ y $(7,)$. Se inicializan con la heurística de He ($\sqrt{2/\text{fan_in}}$), que es la recomendada para capas con ReLU.

Backpropagation

El backpropagation calcula cómo debe cambiar cada peso para reducir el error, usando la regla de la cadena del cálculo diferencial. Se calculan los gradientes capa por capa empezando desde la salida:

```

d3 = clip_grad(o_out - y_enc)          # Error en la capa de salida
d2 = clip_grad((d3 @ W3.T) * (h2_in > 0)) # Error en capa oculta 2
d1 = clip_grad((d2 @ W2.T) * (h1_in > 0)) # Error en capa oculta 1

```

Los pesos se actualizan con SGD con momentum. El momentum acumula los gradientes de iteraciones anteriores para suavizar las actualizaciones:

```

vW = momentum * vW - lr * np.outer(input_layer, delta)
W  = W + vW

```

Dataset y Balanceo

El dataset contiene 142 snippets de código etiquetados manualmente, distribuidos en las 7 clases. Para balancear las clases (algunas tenían más muestras que otras) usamos augmentación gaussiana: a los ejemplos de clases con pocas muestras se les añade ruido gaussiano suave ($\sigma=0.015$) para generar nuevas muestras artificiales sin distorsionar las características originales.

Clase	Complejidad	Muestras originales
0	$O(1)$	22
1	$O(\log n)$	20
2	$O(n)$	25
3	$O(n \log n)$	40
4	$O(n^2)$	42
5	$O(n^3)$	15
6	$O(2^n)$	18

Después del balanceo, todas las clases tienen 42 muestras (las de $O(n^2)$, que es la clase con más ejemplos). El dataset se divide en 80% para entrenamiento y 20% para prueba.

Proceso de Entrenamiento

El entrenamiento sigue estos pasos en cada época: primero se mezcla aleatoriamente el conjunto de entrenamiento. Luego se divide en mini-batches de tamaño 8 usando la estructura Queue. Para cada muestra del batch se hace: forward pass → cálculo del error → backpropagation → actualización de pesos. Después se calculan el accuracy y el loss promedio de la época, se guardan las métricas en la HashTable y se verifica la condición de Early Stopping. Cada 100 épocas se reducen el learning rate por un factor de 0.5.

Al final del entrenamiento se restauran los pesos de la mejor época (la de mayor accuracy). También se aplica Hard Example Mining para identificar las 5 muestras con mayor pérdida.

Resultados del entrenamiento

Los resultados obtenidos en nuestras pruebas fueron los siguientes: accuracy en entrenamiento del 100% (la red memorizó el dataset de entrenamiento), accuracy en prueba (test set) de aproximadamente 93.75% en promedio, entre 100 y 250 épocas antes del Early Stopping, y el learning rate decayendo de 0.005 hasta aproximadamente 0.00125 en la mayoría de las ejecuciones.

Feature Extractor — Extracción de Características del Código

¿Qué es el AST y por qué lo usamos?

El AST (Abstract Syntax Tree) es una representación en forma de árbol de la estructura sintáctica de un programa. Python incluye el módulo ast en su librería estándar, que permite analizar código fuente y recorrer su estructura de forma programática, sin necesidad de ejecutarlo.

Usamos el AST porque nos permite detectar patrones estructurales (bucles anidados, recursividad, ordenamientos, etc.) que son precisamente los que determinan la complejidad algorítmica. No nos importa qué hacen las variables, sino cómo está organizado el código.

Las 20 características extraídas

El extractor analiza el AST y produce un vector de 20 valores numéricos, todos normalizados al rango [0, 1]:

#	Feature	Qué detecta	Normalización
0	for_loops	Cantidad de bucles for	÷ 10
1	while_loops	Cantidad de bucles while	÷ 10
2	max_depth	Profundidad máxima de anidamiento	÷ 5
3	has_recursion	Hay recursividad (sí/no)	0 ó 1
4	recursive_calls	Número de llamadas recursivas	÷ 5
5	has_sort	Uso de sorted() o .sort()	0 ó 1
6	has_binary_search	Patrón de búsqueda binaria	0 ó 1
7	total_calls	Total de llamadas a funciones	÷ 20
8	num_lines	Líneas efectivas de código	÷ 50
9	has_divide_by_2	División entera por 2 (/2)	0 ó 1
10	sequential_loops	Bucles al primer nivel (secuenciales)	÷ 10
11	has_hash_access	Acceso con índice: arr[i]	0 ó 1
12	nested_loop_depth_2	Exactamente profundidad 2	0 ó 1
13	has_sort_plus_loop	Sort + bucle secuencial	0 ó 1
14	recursive_divide	Recursión + división por 2	0 ó 1
15	has_sort_in_function	Sort dentro de función iterativa	0 ó 1
16	loop_count_ratio	Proporción for/(for+while)	0.0 a 1.0
17	has_triple_nest	Profundidad de anidamiento ≥ 3	0 ó 1
18	calls_per_line	Llamadas por línea de código	÷ 2
19	has_two_rec_calls	Exactamente 2 llamadas recursivas	0 ó 1

Ejemplos de vectores de features

Para entender mejor cómo funciona el extractor, veamos algunos ejemplos concretos:

Fibonacci recursivo: las features más relevantes son has_recursion=1.0, has_two_rec_calls=1.0, for_loops=0, while_loops=0. El modelo lo clasifica como $O(2^n)$.

Búsqueda binaria: las features más relevantes son `has_binary_search=1.0`, `has_divide_by_2=1.0`, `while_loops=1`, `loop_count_ratio=0.0`. El modelo lo clasifica como $O(\log n)$.

Bubble sort: las features más relevantes son `for_loops=2`, `max_depth=2`, `nested_loop_depth_2=1.0`. El modelo lo clasifica como $O(n^2)$.

Multiplicación de matrices: las features más relevantes son `for_loops=3`, `max_depth=3`, `has_triple_nest=1.0`. El modelo lo clasifica como $O(n^3)$.

Manejo de errores

Si el código enviado tiene errores sintácticos (`SyntaxError`), el extractor devuelve un vector de ceros para no bloquear el sistema. Es una decisión de diseño práctica: preferimos dar una predicción poco confiable antes que lanzar una excepción que rompa el flujo.

RA3 — Estructuras de Datos Propias

Cola (Queue)

Implementamos una cola FIFO (First In, First Out) desde cero usando una lista de Python internamente. La usamos durante el entrenamiento para gestionar los mini-batches: en cada época, los índices del dataset se dividen en grupos de tamaño 8 y se encolan. Luego se van desencolando y procesando uno por uno.

Operación	Complejidad Temporal	Descripción
<code>enqueue(item)</code>	$O(1)$	Agrega al final con <code>list.append()</code>
<code>dequeue()</code>	$O(n)$	Remueve el primero con <code>list.pop(0)</code>
<code>is_empty()</code>	$O(1)$	Verifica si la lista está vacía
<code>size()</code>	$O(1)$	Retorna <code>len(items)</code>

El `dequeue` tiene complejidad $O(n)$ porque `list.pop(0)` en Python desplaza todos los elementos restantes. Esto no afecta significativamente el rendimiento en nuestro caso porque los batches son pequeños.

Tabla Hash (HashTable)

Implementamos una tabla hash con encadenamiento separado (Separate Chaining) para manejar colisiones. La usamos para almacenar las métricas de entrenamiento: `accuracy` y `loss` de cada época. La clave es algo como `'epoch_42_acc'` y el valor es el `accuracy` en esa época. La tabla tiene 100 buckets por defecto y usa la función hash nativa de Python con módulo.

Operación	Caso promedio	Peor caso	Descripción
set(key, value)	O(1)	O(n)	Inserta o actualiza
get(key)	O(1)	O(n)	Busca por clave
_hash(key)	O(1)	O(1)	Calcula índice del bucket

El peor caso $O(n)$ ocurre cuando todas las claves colisionan en el mismo bucket. En la práctica esto no sucede porque las claves de métricas son bastante distintas entre sí.

batch_loader.py — Generador de Mini-Batches

Este módulo usa la estructura Queue para dividir el dataset en mini-batches. Recibe las features X y las etiquetas y, y devuelve una cola con pares (X_batch, y_batch). Esto permite al MLP procesar el dataset en grupos pequeños en lugar de todo a la vez.

```
def create_batches(X, y, batch_size=16):
    queue = Queue()
    for i in range(0, len(X), batch_size):
        queue.enqueue((X[i:i+batch_size], y[i:i+batch_size]))
    return queue
```

Complejidad temporal: $O(n)$, donde n es el número de muestras del dataset.

RA2 — Algoritmos Implementados

ard Example Mining

Hard Example Mining es una técnica de entrenamiento que identifica los ejemplos que el modelo encontró más difíciles (los que tuvieron mayor pérdida). Lo usamos para reportar qué tipos de algoritmos son más complicados de clasificar.

Implementamos la función get_hard_examples() usando heapq.nlargest(), que internamente usa un heap de tamaño k para encontrar los k elementos máximos de una lista sin ordenarla completamente:

```
def get_hard_examples(losses, k=5):
    indexed_losses = [(loss, idx) for idx, loss in enumerate(losses)]
    return heapq.nlargest(k, indexed_losses)
```

Aspecto	Detalle
Complejidad Temporal	$O(n \log k)$ — n = total de pérdidas, k = ejemplos deseados
Complejidad Espacial	$O(n)$ — por la lista indexada auxiliar
Ventaja sobre <code>sort()</code>	No ordena todo: mantiene un heap de tamaño k mientras itera

Quickselect — Cálculo de Mediana

Implementamos el algoritmo Quickselect para calcular la mediana de un conjunto de datos. Quickselect es una variante de Quicksort que solo explora la partición que contiene el elemento buscado, evitando ordenar completamente el arreglo.

El algoritmo usa la estrategia de partición de Lomuto: toma el último elemento como pivote y lo coloca en su posición final. Luego busca recursivamente solo en la mitad relevante:

```
def quickselect(arr, low, high, k):
    if low == high: return arr[low]

    pivot_index = partition(arr, low, high)

    if pivot_index == k: return arr[k]
    elif pivot_index > k: return quickselect(arr, low, pivot_index-1,
k)
    else: return quickselect(arr, pivot_index+1,
high, k)
```

Caso	Complejidad Temporal	Complejidad Espacial
Promedio	$O(n)$	$O(\log n)$ por recursión
Peor caso	$O(n^2)$	$O(n)$ por recursión desbalanceada
<code>sort()</code> comparativo	$O(n \log n)$	$O(n)$

En el benchmark experimental verificamos que Quickselect es más rápido que `sorted()` para encontrar la mediana, especialmente con n grande. Para $n=100.000$ la diferencia es notoria.

Top-K — Heap vs Ordenamiento Completo

Implementamos dos estrategias para encontrar los K elementos más grandes de una lista y las comparamos experimentalmente:

`top_k_heap()`: usa `heapq.nlargest()` con complejidad $O(n \log k)$. Mantiene un min-heap de tamaño k mientras recorre la lista.

`top_k_sort()`: ordena todo el arreglo y toma los primeros k , con complejidad $O(n \log n)$. Simple pero menos eficiente cuando $k \ll n$.

n (tamaño)	Heap Top-K (s)	Sort Top-K (s)	Speedup aprox.
1.000	0.0003	0.0009	~3x
5.000	0.0007	0.0045	~6x
10.000	0.0012	0.0099	~8x
50.000	0.0054	0.0512	~9x
100.000	0.0099	0.1084	~11x

Como era de esperarse teóricamente, el Heap es significativamente más eficiente cuando k es pequeño respecto a n . Para $k=10$ y $n=100.000$, la diferencia es de casi 11x.

API REST — FastAPI

La API fue desarrollada con FastAPI y expone todos los servicios del sistema. Actúa como intermediario entre el bot de Telegram y los módulos internos.

Endpoint	Método	Descripción
GET /	GET	Verificación de disponibilidad
GET /status	GET	Estado del modelo (trained/not_trained) y accuracy
POST /train_complexity	POST	Inicia el entrenamiento del MLP
POST /analyze	POST	Clasifica la complejidad de un código enviado
GET /analyze/plot	GET	Genera imagen PNG con la gráfica de complejidad
GET /complexity_metrics	GET	Historial de accuracy y loss por época
GET /hard-examples	GET	Los 5 ejemplos con mayor pérdida
GET /benchmark	GET	Resultado del benchmark Heap vs Sort
GET /complexity	GET	Complejidades teóricas de los módulos del proyecto

Todos los endpoints retornan JSON excepto `/analyze/plot`, que retorna una imagen PNG generada en memoria con Matplotlib (usando el backend Agg, sin interfaz gráfica).

Bot de Telegram

El bot fue desarrollado con la librería `python-telegram-bot` y funciona como interfaz principal del sistema. Soporta tanto comandos explícitos como detección automática de código pegado en el chat.

Comando	Descripción
<code>/start</code>	Mensaje de bienvenida
<code>/help</code>	Lista de comandos disponibles
<code>/train_complexity</code>	Entrena el MLP y muestra resultados
<code>/analyze <código></code>	Clasifica la complejidad y muestra gráfica
<code>/status</code>	Estado del modelo
<code>/complexity_metrics</code>	Resumen de métricas por época
<code>/hardexamples</code>	Los ejemplos más difíciles del entrenamiento
<code>/benchmark</code>	Resultado del benchmark Heap vs Sort
<code>/complexity</code>	Complejidades teóricas de los módulos
<code>[texto libre]</code>	Detecta automáticamente si es código Python

Una funcionalidad que nos gustó especialmente es la detección automática de código: si el usuario pega un fragmento con palabras clave como `'def'`, `'for'` o `'while'`, el bot lo detecta y lo analiza sin necesidad de usar el comando `/analyze`. Tras cada análisis, también envía una imagen PNG generada dinámicamente con la curva de la complejidad detectada y un panel comparativo.

Complexity Plotter — Generación de Gráficas

El módulo `complexity_plotter.py` genera gráficas de complejidad algorítmica como imágenes PNG en memoria (sin escribir archivos en disco). Cada gráfica tiene dos paneles: el izquierdo muestra la curva de la complejidad detectada con su etiqueta y una anotación; el derecho muestra una comparativa de todas las complejidades con la detectada resaltada.

Complejidad	Rango del eje Y	Observación
$O(1)$	0 a 5	Línea horizontal
$O(\log n)$	0 a 12	Crece muy lentamente
$O(n)$	0 a 110	Lineal
$O(n \log n)$	0 a 800	Más rápida que lineal
$O(n^2)$	0 a 12.000	Crecimiento notable
$O(n^3)$	0 a 1.200.000	Rango muy alto
$O(2^n)$	0 a 1.200	Solo n de 1 a 10

Para $O(2^n)$ usamos un rango de n de 1 a 10 porque con $n=100$ los valores serían astronómicamente grandes. Para $O(n^3)$ el eje Y llega hasta 1.200.000 porque es la escala natural del crecimiento cúbico con $n=100$.

Suite de Pruebas Unitarias

Desarrollamos pruebas unitarias para todos los módulos principales usando pytest. Cada módulo tiene su propio archivo de tests con casos normales (happy path) y casos límite (edge cases).

Archivo de tests	Módulo testeado	Casos cubiertos
test_batch_loader.py	batch_loader.py	10 casos: batches vacíos, incompletos, alineación X-y
test_complexity_model_service.py	complexity_model_service.py	12 casos: ReLU, Softmax, Gradient Clipping
test_complexity_plotter.py	complexity_plotter.py	14 casos: PNG válido, todas las complejidades, inválidas
test_encoding.py	encoding.py	10 casos: One-Hot para distintos tipos de entrada
test_feature_extractor.py	feature_extractor.py	50+ casos: cada feature individual y huellas por complejidad
test_hard_mining.py	hard_mining.py	15 casos: valores extremos, duplicados, negativos
test_hash_table.py	hash_table.py	18 casos: inserción, actualización, colisiones
test_queue.py	queue.py	17 casos: FIFO, vacío, tipos distintos

test_quickselect.py	quickselect.py	15 casos: partition, quickselect, find_median
test_top_k.py	top_k.py	10 casos: heap vs sort, duplicados, negativos

Para ejecutar todos los tests: `python3 -m pytest tests/ -v`

Para ejecutar un archivo específico: `python3 -m pytest tests/test_feature_extractor.py -v`

Análisis de Complejidad del Sistema

MLP — Forward Pass

El forward pass procesa una sola muestra a la vez. Las operaciones dominantes son los productos matriciales: Capa 1: $X (1 \times 20) @ W1 (20 \times 128) \rightarrow O(20 \times 128) = O(2560)$; Capa 2: $h1 (1 \times 128) @ W2 (128 \times 64) \rightarrow O(128 \times 64) = O(8192)$; Capa 3: $h2 (1 \times 64) @ W3 (64 \times 7) \rightarrow O(64 \times 7) = O(448)$. La complejidad total del forward pass es $O(\text{input} \times h1 + h1 \times h2 + h2 \times \text{output}) = O(B \times h \times \text{output})$ donde B es el batch size.

Extractor de Features

Recorre el AST del código una sola vez (con `ast.walk`), más una pasada adicional para detectar recursividad. Si el código tiene N nodos en el AST, la complejidad es $O(N)$. En la práctica N es aproximadamente igual al número de líneas de código, así que es $O(\text{líneas})$.

Tabla Hash

Las operaciones de set y get son $O(1)$ en promedio con el encadenamiento separado. El peor caso es $O(n)$ si todas las claves caen en el mismo bucket, lo cual en la práctica no ocurre con nuestras claves de métricas.

Queue

Enqueue es $O(1)$. Dequeue es $O(n)$ por el `list.pop(0)` de Python. Una mejora posible sería usar `collections.deque` que tiene dequeue $O(1)$, pero decidimos no hacerlo para mantener la implementación propia y simple.

Hard Example Mining

$O(n \log k)$ donde n es el total de pérdidas registradas y $k=5$ los ejemplos deseados. Esto es mucho mejor que ordenar toda la lista en $O(n \log n)$ cuando $k \ll n$.

Conclusiones

Logramos construir un sistema funcional de extremo a extremo que clasifica la complejidad algorítmica de código Python con una precisión de prueba de aproximadamente el 93.75%. El sistema integra de manera coherente los tres resultados de aprendizaje del curso:

RA1: una red neuronal MLP implementada completamente desde cero con NumPy, con forward pass, backpropagation, SGD con momentum, early stopping, learning rate decay y gradient clipping.

RA2: algoritmos clásicos de análisis de complejidad (Quickselect, Top-K con Heap) usados internamente para el procesamiento eficiente de datos de entrenamiento.

RA3: estructuras de datos propias (Queue, HashTable) integradas al pipeline de entrenamiento para el manejo de mini-batches y el almacenamiento de métricas.

El proyecto demuestra que es posible construir un sistema de machine learning sin depender de frameworks especializados, siempre que se entienda bien la matemática y la algorítmica detrás de cada componente. Esa comprensión profunda es, al final, el objetivo principal de la asignatura.

Referencias

- Python Software Foundation. (2024). ast — Abstract Syntax Trees. Python 3.12 Documentation.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.
- python-telegram-bot contributors. (2024). python-telegram-bot v22 Documentation.
- Tiangolo, S. (2024). FastAPI Documentation. <https://fastapi.tiangolo.com>
- NumPy developers. (2024). NumPy Documentation. <https://numpy.org/doc/>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95.